

# 面向 LLM+OR 的 Operational Problem Complexity: 从约束耦合理论到可验证复杂度定义

## 执行摘要

截至 2026 年 8 月，我对现有 LLM+OR、自动优化建模、约束建模、CSP 结构复杂度和工业 MILP benchmark 文献的综合判断是：目前还没有一个被 OR 社区广泛接受、**problem-intrinsic**、**model-independent**，并能像 **computational complexity** 那样系统刻画“一个完整 operational problem 有多难”的 **formal definition**。现有工作已经分别碰到了这个问题的不同侧面，但尚未把它们统一起来。OPT-Engine 用 problem scale 和 constraint augmentation 控制难度，并发现 solver-integrated reasoning 的主要瓶颈在 constraint formulation；ORAgentBench 明确提出六维 construction-time difficulty rubric；LEAN-LLM-OPT 使用 input/output scale 等经验量来研究 modeling degradation；MILP-Bench 则进一步把 **semantic constraint type** 与 **loop-based structural scaffold** 区分为两个正交的建模维度。这些工作共同构成了我们定义 operational complexity 的直接理论与实证起点。<sup>1</sup>

更重要的是，你最初关于 OPT-Engine 的判断需要作一个精确修正。OPT-Engine 确实写道：在 Solver-Integrated Reasoning 中，如果模型正确捕获 constraints/objective 的 logical grounding，外部 solver 就保证 optimality。<sup>2</sup> 但同一篇论文随后明确承认，这个结论属于它所研究的 **exact-solution paradigm**；对于 large-scale NP-hard problems，exact modeling/solving 可能 computationally prohibitive，因此实际工作往往需要 heuristics，并建议未来显式评测 optimality gap、runtime 和 time-budget robustness。<sup>3</sup> Gurobi 的官方文档同样区分 **OPTIMAL**、time-limit termination、incumbent、best bound 和 MIP gap：一个正确 MILP 模型在有限资源下完全可能只得到 feasible incumbent 而没有 optimality certificate。<sup>4</sup>

因此，你的命题：

“对 LLM 来说，正确表达全部 feasibility conditions，比把一个已经正确 formalize 的模型交给 solver 求 optimum 更难”

很可能在当前大多数 NL-to-Opt benchmark 的 exact-solver regime 中成立，但不应直接提升为所有 OR 问题的普遍定理。更准确且可发表的研究假设应当是：

For solver-integrated LLM systems on exact-solvable benchmark regimes,  $C_{\text{model}}$  explains substantially more

然后通过 gold-model intervention 实验去决定这个命题的 **适用边界**，而不是先把 computational difficulty 从定义中删掉。

我建议从现在开始明确区分三个概念：

$$C_{\text{OPC}}(I) = (C_{\text{OMC}}(I), C_{\text{RCB}}(I; \Pi))$$

其中：

- **Operational Problem Complexity, OPC** 是总概念；
- **Operational Modeling Complexity, OMC** 是我们真正重点研究的、尽可能 problem-intrinsic 的 semantic/structural modeling burden；

- **Residual Computational Burden, RCB** 是在 gold model 已经给定后，在标准 solver protocol II 下仍然存在的求解负担。

第一篇论文最值得集中攻击的是 **OMC**，而不是一开始强行把所有问题压成一个 scalar OPC。

更具体地，我建议第一版 OMC 保持为 vector：

$$\mathbf{C}_{\text{OMC}} = \left[ \underbrace{\mathbf{S}}_{\text{semantic obligation load}}, \underbrace{\mathbf{R}}_{\text{scope/scaffold structure}}, \underbrace{\mathbf{K}_{\text{local}}}_{\text{direct coupling}}, \underbrace{\mathbf{K}_{\text{global}}}_{\text{global interaction}}, \underbrace{\mathbf{D}}_{\text{dynamic/information structure}} \right]$$

而不是把 LLM accuracy 本身定义成 complexity。LLM accuracy 应该作为 **criterion validity**——验证 **complexity** 是否有效的外部结果变量，而不是 **complexity** 的定义。否则同一道问题在 GPT-5.x、未来模型和专门训练的 OR agent 上会得到不同 “complexity”，就不再是 problem-intrinsic measure。

我认为目前最有希望形成论文核心的新结构是：

### Semantic Obligation Hypergraph (SOH)

但必须强调：“**Semantic Obligation Hypergraph**” 并不是现有文献中的标准术语，而是我们基于 **CSP constraint hypergraph**、**MIPLIB-NL semantic/scaffold decomposition** 和最新 **CIR rule-to-constraint** 工作提出的研究构造。CSP 理论已经证明，约束的 graph/hypergraph incidence structure 与 tractability 有深刻关系；MIPLIB-NL 又从工业优化建模角度指出，constraint family 的难度不能只看 semantic type，还取决于 loop/scaffold 如何复制、聚合、递归和耦合约束。<sup>5</sup>

这恰好给了我们一条比 “constraint 数量越多越难” 强得多的路线：

difficulty /

下面我先把整个理论基础从数学上建立起来，再给出一个完整例题，把各项 coupling 指标逐项算出来，最后设计论文实验。

## 文献地图、已有复杂度概念与真正的研究缺口

### 现有 LLM+OR 文献究竟已经做到哪里

自动 optimization modeling 最早一批工作主要研究把自然语言映射成 variables、objective 和 constraints。NL4Opt 一系工作已经指出，优化问题的自然语言输入具有 document-level、compositionality 和 ambiguity 等特点，因此它本质上不仅是公式抄写，而包含 semantic parsing。<sup>6</sup> OptiMUS 随后把 LLM 与 (MI)LP solver、code generation 和 debugging 结合，进一步把任务推向 solver-integrated workflow。<sup>7</sup> ORLM/IndustryOR 则将训练和 benchmark 扩展到更接近 industrial OR 的场景。<sup>8</sup>

但这些工作中的 “complexity” 多数并不是你现在想研究的 complexity。一个尤其值得写进 future paper related-work critique 的例子是 2025 年 optimization-modeling survey：其 benchmark-level

“Complexity” 指标实际上是先让模型生成 formulation，然后用生成模型中的 **number of variables + number of constraints** 作为 complexity indicator。<sup>9</sup> 这对描述 benchmark 大小有用，但并不满足你的目标，因为它既 **model-dependent**，又会受到不同等价 formulation 的影响。OptiBench/ORGEval 一类工作恰恰表明 optimization formulations 存在 equivalence 问题，因此 solver-level representation 本身不能简单等同于 problem-intrinsic semantics。<sup>10</sup>

| 工作                         | 它实际上在测什么                                           | 对我们最有价值的部分                                                                             | 仍然缺什么                                                                 |
|----------------------------|----------------------------------------------------|----------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| NL4Opt                     | NL→optimization formulation                        | 把 optimization modeling 明确变成 semantic parsing 问题                                       | 没有 problem-intrinsic complexity theory <sup>6</sup>                   |
| OptiMUS                    | formulation + solver code + debug                  | solver-integrated workflow                                                             | difficulty 不是主要理论对象 <sup>7</sup>                                      |
| ORLM / IndustryOR          | industrial modeling accuracy                       | 更真实 problem distribution                                                               | difficulty 主要作为 benchmark 分层而非数学结构 <sup>8</sup>                       |
| CP-Bench                   | NL→MiniZinc/CPMpy/CP-SAT models                    | 把 constraint modeling 扩展到 combinatorial/global-constraint setting; 最高配置仍约 70% accuracy | 没有 intrinsic coupling metric <sup>11</sup>                            |
| OPT-Engine                 | scale-controlled LP/ MIP + constraint augmentation | 直接发现 constraints 是 SIR 主要瓶颈                                                            | complexity 仍主要由 problem scale/ template augmentation 控制 <sup>12</sup> |
| LEAN-LLM-OPT               | large-scale modeling                               | input length、output variables 与 accuracy degradation; 含 NRM                            | 这些仍是 scale proxies, 而不是语义耦合定义 <sup>13</sup>                           |
| MIPLIB-NL                  | industrial MILP NL↔formal model                    | <b>semantic type × loop scaffold</b> , 直接指出 coupling/indexing 结构的重要性                   | 尚未把结构压成可预测 LLM difficulty 的数学 measure <sup>14</sup>                   |
| CIR / R2C / ORCOpt         | operational rule→constraint                        | 明确加入 semantic intermediate representation, 处理 composite operational rules              | 目标是提升生成准确率, 不是定义 complexity <sup>15</sup>                             |
| ORAgentBench               | full end-to-end OR agent                           | 六维 difficulty rubric; 区分 modeling strategy 与 solving strategy                          | 六维并非严格数学定义, 而且有共线性 <sup>16</sup>                                      |
| ConstraintBench            | LLM 不用 solver, 直接生成 solution                       | feasibility 与 optimality 可明显解耦                                                         | 属于 direct-solving, 而非 solver-integrated modeling <sup>17</sup>        |
| MIPLIB / solver benchmarks | solver-side computational hardness                 | 给 residual computational burden 提供标准化参照                                                | 没有 NL/semantic modeling burden <sup>18</sup>                          |

## OPT-Engine、ORAgentBench 的三个具体问题

OPT-Engine 是否调用 solver? ——要区分 paradigm。

它的 **Solver-Integrated Reasoning, SIR** 确实要求模型生成 executable solver code, 然后交给 external solver 执行; 因此最终数值 optimization 不是 LLM 自己心算。相反, Pure-Text Reasoning 是模型直接推理; CompPTR 虽然允许 Python、SciPy、`itertools` 一类 computational primitives, 但论文明确禁止它调用 external global optimization solver。 <sup>2</sup>

所以:

$$\text{OPT-Engine SIR} = \text{LLM formulation/code} + \text{external solver}.$$

这正是为什么它能观察到 formulation correctness 成为主要瓶颈。 <sup>12</sup>

**ORAgentBench 的 end-to-end 绝不是“把题发送给 LLM, 让它直接回答最优解”。**

任务包含 natural-language brief、multiple files、configuration artifacts 和 submission schema; agent 必须解释 artifacts、选择 modeling strategy 和 solving strategy、编写并运行程序, 再输出 decision artifact, 由 hidden validator 检查 schema、hard feasibility 和 objective quality。 <sup>19</sup> ORAgentBench 甚至明确将内部过程写成 latent modeling strategy  $\phi_{\theta,\tau}$  和 solving strategy  $\psi_{\theta,\tau}$ , 因此它正好代表比“直接答题”更完整的 operational workflow。 <sup>20</sup>

**你对六维 rubric 重叠的感觉是对的, 而且论文自己其实已经暴露了这个问题。**

六个维度是:

{solving strategy, formulation structure, constraint coupling, dynamic structure, data scale, problem understanding}

论文把它们作为 construction-time rubric, 用于 prospective difficulty design, 但正文并没有给出像 treewidth 那样彼此严格区分的数学定义。 <sup>16</sup> 更关键的是, 它在 appendix 的 multivariate analysis 中明确指出: constraint coupling 的 joint coefficient 较小, **部分原因就是它与 formulation structure 和 dynamic structure 存在 correlation。** <sup>21</sup>

所以我们不应该直接复制 ORAgentBench 六维体系。我们应该做的是把这些概念重新拆成更接近 **causal stages** 的结构:

|                                                    |
|----------------------------------------------------|
| What rules? → semantic obligation load             |
| How instantiated? → scope/scaffold complexity      |
| How interacting? → constraint coupling             |
| How state evolves? → dynamic/information structure |
| How represented in files? → data-interface burden  |
| How hard after gold model? → residual solve burden |

其中你的第一篇工作可以故意 **把 data-interface burden 当 control variable, 而不是核心 OMC component**, 因为你真正想回答的是 modeling logic 本身为什么难。

## MIPLIB-NL 是目前最关键的直接前驱

MIPLIB-NL 对我们特别重要。它从 MIPLIB 2017 的真实 MILP 反向构造 223 个自然语言实例, 并把 natural-language specification、formal mathematical model 和原始数据对齐。其规模中位数达到 2,183 variables 和 1,388 constraints, 并且保留工业模型的 index/constraint-family structure。 <sup>22</sup>

它提出一个极其接近我们目标的观察：

$$\text{Constraint Family} \approx (\text{Semantic Constraint Type, Loop-Based Structural Scaffold})$$

semantic type 描述“这个 rule 在数学上要求什么”，而 scaffold 描述“这个 rule 如何跨 index domains 生成、复制、聚合和耦合”。作者强调这两个维度大体正交，而且仅覆盖 semantic constraint types 会严重低估真实 MILP 的 structural richness。 <sup>23</sup>

它观察到的 scaffold 包括 nested loops、subset-indexed loops、temporal/recursive coupling、cyclic/modular indexing、sliding-window aggregation、all-pairs/complete-graph indexing，以及 commodity/scenario extra-dimension replication。 <sup>24</sup> 特别是 all-pairs scheduling 会产生接近 complete-graph 的 pairwise structure，constraint family 可以随  $O(n^2)$  增长。 <sup>25</sup>

这实际上给我们的定义指出了方向：

**constraint complexity 不应从“有多少条约束”出发，而应从“有多少种 semantic obligations、这些 obligations 如何被 structural scaffolds 实例化，以及它们怎样通过共享 decisions 形成 interaction topology”出发。**

## 约束耦合的数学基础：从 CSP 到 Semantic Obligation Hypergraph

### 先理解 CSP

Constraint Satisfaction Problem 可以抽象成：

$$\mathcal{P} = (X, D, C),$$

其中

$$X = \{x_1, \dots, x_n\}$$

是变量集合，

$$D_i$$

是变量  $x_i$  的 domain，而每个 constraint

$$c_j = (S_j, R_j)$$

包含一个变量 scope

$$S_j \subseteq X$$

以及允许这些变量组合取值的 relation  $R_j$ 。

例如：

$$x_1 + x_2 \leq 10$$

的 scope 是

$$S = \{x_1, x_2\}.$$

而

$$x_1 + x_2 + x_3 + x_4 = 1$$

的 scope 是

$$S = \{x_1, x_2, x_3, x_4\}.$$

CSP 理论的关键发现不是“constraint 越多就越难”，而是 **constraint scopes 如何相互连接** 对 tractability 有决定性意义。对于 binary CSP, constraint graph 的 treewidth 是最重要的 structural complexity parameters 之一；对于 unbounded-arity constraints, 普通 graph treewidth 不再足够，需要 hypergraph width, 例如 fractional hypertree width 或更一般的 submodular width。 26

这正是为什么 CSP 是我们研究 constraint coupling 最好的理论母体之一。

## Hypergraph 为什么比普通 graph 更自然

普通 graph:

$$G = (V, E)$$

中的一条 edge 只能连接两个 vertices:

$$e = \{u, v\}.$$

hypergraph:

$$H = (V, \mathcal{E})$$

则允许:

$$e \subseteq V$$

包含任意数量 vertices。

例如一个 capacity rule:

$$x_1 + x_2 + x_3 + x_4 \leq C$$

自然对应:

$$e = \{x_1, x_2, x_3, x_4\},$$

而不是人为拆成六条 pairwise graph edges。

这非常重要，因为一个 operational rule 往往天然同时涉及很多 decisions, 例如 facility capacity:

$$\sum_c q_{fct} \leq K_f y_f + o_{ft},$$

其语义不是一组互不相关的 pairwise relations, 而是一个 **joint feasibility obligation**。

## Semantic Obligation Hypergraph 是什么

这里开始是本项目建议的新定义，而不是现有文献的标准术语。

先定义完整 operational instance:

$$I = (T, D, R),$$

其中:

- $T$ : 自然语言 task description;
- $D$ : structured/unstructured data;
- $R$ : business rules.

我们不直接从 solver variables 建 graph, 而先定义:

$$V^S = \{v_1, \dots, v_n\},$$

称为 **Canonical Semantic Decision Entities**。

一个 entity 表示 business problem 中必须决定或追踪的一个规范化决策概念, 例如:

$$\begin{aligned} Y(f) &: \text{facility } f \text{ 是否开放} \\ X(f, c) &: \text{customer } c \text{ 是否分配给 } f \\ Q(f, c, t) &: \text{时期 } t \text{ 从 } f \text{ 向 } c \text{ 发货量} \\ P(f, t) &: \text{production amount} \\ I(f, t) &: \text{inventory} \\ B(c, t) &: \text{backlog.} \end{aligned}$$

“canonical” 最重要的目的, 是尽量对 **solver formulation choice** 不敏感。例如 Big- $M$  auxiliary variable、subtour elimination auxiliary variable、slack variable 不应该因为某一种 formulation 使用了它们, 就增加 problem intrinsic complexity。

这也是整个项目最困难的一点之一: **canonical semantic entity 不可能仅靠 solver code 唯一恢复**。Route-based、arc-based、path-based formulations 有时会给同一现实 decision 提供非常不同的数学 representation。因此 canonicalization 必须以 **business decision ontology / gold semantic annotation** 为准, 而不是以 generated variable names 为准。这个 invariance 本身以后应成为实验验证对象。

接下来定义 **Semantic Obligation**:

一个可以独立检查其是否被正确建模的 business requirement。

例如:

“每个 customer 必须且只能分配给一个 facility。”

是一项 obligation:

$$r_1 : \sum_f X(f, c) = 1, \quad \forall c.$$

而：

“只能把 customer 分配给已经开放的 facility。”

是另一项：

$$r_2 : X(f, c) \leq Y(f).$$

每项 obligation  $r_j$  产生一个 hyperedge：

$$e_j = \{v \in V^S : v \text{ is jointly referenced by } r_j\}.$$

于是：

$$H_{\text{SOH}} = (V^S, \mathcal{E}^S)$$

就是 Semantic Obligation Hypergraph。

注意这里最有价值的设计是：**node 是 operational decision entity，而 hyperedge 是 business obligation，不是 solver-level row。**

这种结构继承了 CSP constraint-hypergraph 的理论思想，但把“变量”提升到了 semantic decision level；与此同时，最新 CIR/R2C 工作也独立地说明了在自然语言和数学 constraint 之间加入 structured semantic layer 的合理性：CIR 就是通过 constraint archetypes 和 candidate modeling paradigms 把 operational-rule semantics 与具体 mathematical instantiation 解耦。<sup>15</sup>

## Family level 与 grounded-instance level 必须区分

这里有一个非常关键的细节。

考虑：

“相邻两个 period 的生产变化不得超过 20。”

在 semantic family level 上，它只涉及一个 entity family：

$$P(f, t),$$

所以 hyperedge 似乎是：

$$\{P\}.$$

但实际它产生：

$$|F|(|T| - 1)$$

个跨时间 interaction：

$$|P_{ft} - P_{f,t-1}| \leq R_f.$$

所以我们实际上需要两个 hypergraph：



$H^S =$  schema/family-level SOH

与

$H^G =$  grounded-instance SOH.

family-level 告诉我们：

有哪些概念、哪些 rule families、它们语义上如何相互作用。

grounded-level 告诉我们：

给定具体  $F, C, T, \Omega, \dots$ ，这些 rules 展开以后形成怎样的实际 interaction topology。

这恰好对应 MIPLIB-NL 的 observation：同一个 semantic atom 在不同 loop/index scaffold 下可以产生完全不同的 structural difficulty。 <sup>27</sup>

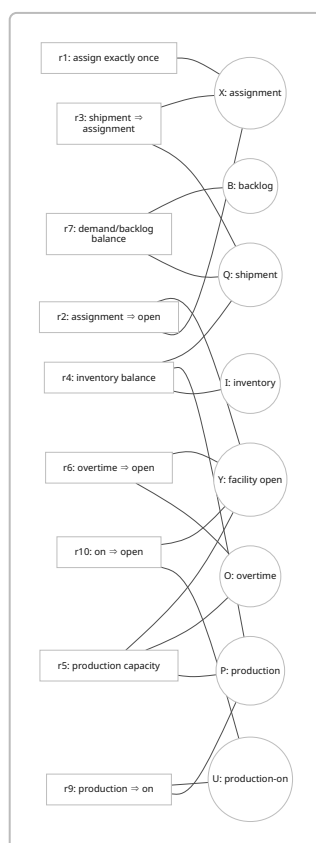
因此以后不要把下面两种 “level” 混淆：

Abstraction level:  $H^S$  vs.  $H^G$

Coupling order: local/direct coupling vs. global/propagated coupling

## Semantic hypergraph 的直观图

下面这张 Mermaid 图用 **incidence representation** 画 hypergraph：方框是 obligations，圆点是 semantic decision entities。一个 obligation 连到多个 entity，就表示一个 hyperedge。



## Mean overlap：最简单的直接耦合

设两个 obligation scopes：

$$e_i, e_j.$$

最直观的 overlap 是：

$$|e_i \cap e_j|.$$

但它偏向 large-scope constraints，因此更合理的是 Jaccard：

$$J_{ij} = \frac{|e_i \cap e_j|}{|e_i \cup e_j|}$$

范围：

$$0 \leq J_{ij} \leq 1.$$

例如：

$$e_i = \{P, O, Y\}, \quad e_j = \{O, Y\},$$

则：

$$J_{ij} = \frac{2}{3}.$$

如果完全无共享 decision：

$$J = 0.$$

如果 scopes 完全相同：

$$J = 1.$$

然后定义：

$$\bar{J} = \frac{2}{m(m-1)} \sum_{i < j} J_{ij}$$

作为 **mean overlap**。

但我建议不要只报告  $\bar{J}$ ，而拆成：

$$D_{\text{ov}} = \frac{\#\{(i, j) : J_{ij} > 0\}}{\binom{m}{2}}$$

和

$$J^+ = \frac{\sum_{i < j : J_{ij} > 0} J_{ij}}{\#\{(i, j) : J_{ij} > 0\}}.$$

其中：

- $D_{ov}$ ：有多少 rule pairs 发生 coupling；
- $J^+$ ：一旦发生 coupling，coupling 有多强。

并且：

$$\bar{J} = D_{ov} J^+.$$

这三个量非常容易解释，也是第一篇论文里最值得保留的 coupling metrics。

## Spectral radius 与 normalized spectral coupling

把 obligations 当作 graph nodes，然后：

$$W_{ij} = J_{ij}, \quad W_{ii} = 0.$$

得到一个 weighted obligation-overlap graph：

$$G_O = (R, W).$$

其 spectral radius：

$$\rho(W) = \max_i |\lambda_i(W)|$$

即 eigenvalues 的最大绝对值。

因为  $W$  是 nonnegative symmetric matrix，在我们的情况中它的最大 eigenvalue 就可以理解为 network 中最强的 **global reinforcement mode**。

直觉上：

- mean overlap 只看“平均一对 constraints 重叠多少”；
- spectral radius 会受到 **dense clusters、high-degree hubs、重复 coupling paths** 的共同影响。

例如一个 star graph 和很多互不相连的小 pair，即便平均 density 类似，它们的 spectral structure 也可能不同。

我建议定义：

$$C_{\text{spec}} = \frac{\rho(W)}{m-1}$$

因为对于  $0 \leq W_{ij} \leq 1$ ：

$$\rho(W) \leq m-1,$$

所以：

$$0 \leq C_{\text{spec}} \leq 1.$$

但这里必须非常谨慎：

**“normalized spectral coupling” 不是已有 OR/CSP 社区标准指标，而是我们为这个项目提出的 candidate metric。**

而且实践中应同时保存：

$$\rho(W) \text{ 和 } C_{\text{spec}},$$

因为 normalization 可能隐藏随规模增长的绝对 interaction burden。

### **Treewidth：目前 coupling 指标里理论基础最强的一项**

由 hypergraph  $H$  构造 **primal graph**：

$$G_P = (V, E_P),$$

若两个 semantic entities  $u, v$  至少共同出现在一个 obligation 中，则：

$$(u, v) \in E_P.$$

tree decomposition 是一棵 tree  $T$ ，其中每个 tree node 带有一个 bag：

$$B_t \subseteq V.$$

它必须满足：

$$\bigcup_t B_t = V,$$

每条 graph edge 的两个 endpoints 至少同时出现在一个 bag 中，并且包含同一个 vertex 的 bags 在 decomposition tree 上必须连通。tree decomposition 的 width 为：

$$\max_t |B_t| - 1.$$

treewidth：

$$tw(G) = \min_{\text{tree decompositions}} \left( \max_t |B_t| - 1 \right).$$

这是 graph theory/CSP 中非常成熟的 structural parameter；对于 binary CSP，constraint graph treewidth 与 computational tractability 有直接理论联系；对于高 arity CSP，则进一步需要 hypertree-type widths。

<sup>26</sup> 对固定 small treewidth，已有经典算法能够构造相应 decomposition；treewidth 本身的精确计算在一般图上并不是简单问题，因此大规模实验中通常需要 exact/FPT algorithms 或近似/heuristic decomposition。

<sup>28</sup>

最重要的直觉是：

**treewidth 测的是：为了把一个全局 interaction network 拆成 tree-like pieces，最少需要保留多大的“接口”。**

几个非常好的参照：

$$\begin{aligned} tw(\text{tree}) &= 1, \\ tw(\text{cycle}) &= 2, \\ tw(K_n) &= n - 1. \end{aligned}$$

所以一个 constraint system 即使 constraints 很多，只要它沿一条 chain/tree 局部连接，treewidth 仍然可能很低。

反之，一个 all-pairs scheduling model 的 primal interaction 可能接近 clique，此时 treewidth 很高。这与 MIPLIB-NL 特别强调 all-pairs/complete-graph scaffolds 的原因高度一致。 <sup>25</sup>

可以定义相对指标：

$$C_{\text{tw}} = \frac{tw(G_P)}{|V| - 1}$$

但第一篇论文中应同时报告：

$$tw(G_P) \text{ 和 } C_{\text{tw}}.$$

我甚至更推荐预测模型使用：

$$\log(1 + tw)$$

并把 graph size 单独作为 covariate，而不是完全依赖 normalized treewidth。

还需要注意：一个 arity 很大的 hyperedge 在 primal graph 中会自动形成 clique，所以 primal treewidth 有可能把“一项大型 global constraint”夸张成极高 coupling。CSP 理论正是因为这个问题，才发展出了 fractional hypertree width 和 submodular width。 <sup>26</sup> 因此论文路线应当是：

$$v1: \text{primal treewidth} \rightarrow v2: \text{hypergraph-specific widths}.$$

不要第一篇论文就把 fractional hypertree width 全塞进去。

## Hubness

定义每个 semantic entity 的 incidence degree：

$$d_H(v) = |\{e_j : v \in e_j\}|.$$

最简单的 hubness：

$$C_{\text{hub}} = \frac{\max_v d_H(v)}{m}.$$

例如一个 `facility_open` variable family 如果同时控制 assignment、production、overtime、staffing、budget、routing activation，那么它会成为 hub。

Hub 的 operational 意义很好解释：

一个 semantic grounding error 发生在 hub entity 上，可能同时污染很多 obligations。

但是 hubness 与 spectral radius 会有相关性，所以我建议把它定位为 **diagnostic metric**，而不是第一版核心独立 dimension。

## Cycle density

对 obligation-overlap graph，只保留：

$$J_{ij} > 0$$

的边，得到 unweighted support graph。

设：

- $q$ : nodes 数；
- $M$ : edges 数；
- $c$ : connected components 数。

它的 cyclomatic number：

$$\mu = M - q + c.$$

如果 graph 是 forest：

$$\mu = 0.$$

于是我们可以定义：

$$C_{\text{cyc}} = \frac{\mu}{\max(1, M)}.$$

这可以理解为：

overlap network 中有多少 interaction edges 是“超出 spanning forest 所需要的额外连接”。

这里的 cycle **不是逻辑矛盾，也不是 solver cycle**，只是表示：

obligations 之间存在多个闭合的 dependency paths。

和 spectral/hubness 一样，这也是我们提出的 diagnostic definition，而不是已有标准。

## bounded-treewidth 到底告诉我们什么

你可以把一个 low-treewidth problem 想象成：

“虽然整个问题很大，但任何时候只需要同时考虑一个很小的 boundary，就可以把局部结果向外传递。”

high-treewidth 则更接近：

“无论怎样拆，很多 decisions 都必须同时保持一致。”

这与我们想要表达的 semantic coupling 极其吻合。

但需要牢记：

$$\text{treewidth/coupling} / \quad = \text{computational}$$

treewidth 是 structural parameter。它在 CSP 中有真正的 tractability theory，因此比普通 graph density 更有理论地位，但我们把它用来预测 **LLM formulation failure** 仍然是一个新的 empirical hypothesis，而不是现成 theorem。 <sup>26</sup>

## 一个更清晰的 Operational Complexity 定义框架

### 不要一开始追求一个 scalar

我目前最推荐的 conceptual hierarchy 是：

$$\text{Operational Problem Complexity} = \text{Operational Modeling Complexity} + \text{Residual Computational Burden}$$

更严格地写成 vector：

$$\mathbf{C}_{\text{OPC}}(I) = [\mathbf{C}_{\text{OMC}}(I), \mathbf{C}_{\text{RCB}}(I; \Pi)] .$$

其中  $\Pi$  是预先固定的 solver protocol。

这是因为 “solver computational difficulty” 如果用 wall-clock runtime 测量，本质上会受到：

solver, formulation, hardware, parameters, time limit

影响，因此严格说它不能和 semantic problem-intrinsic complexity 放在完全相同的 epistemic level 上。Gurobi 官方也区分 runtime、deterministic work、MIP gap、best bound、node limits 等 solver diagnostics。 <sup>29</sup>

所以 RCB 可以定义为一个 **standardized empirical axis**：

$$\mathbf{C}_{\text{RCB}} = [\log(1 + \text{work}), \log(1 + \text{nodes}), \text{time-to-feasible}, \text{time-to-target-gap}, \text{final gap}] .$$

而真正 problem-intrinsic 的研究重点放在 OMC。

### 推荐的第一版 OMC vector

我建议：

$$\mathbf{C}_{\text{OMC}}(I) = [\mathbf{S}(I), \mathbf{R}(I), \mathbf{K}_L(I), \mathbf{K}_G(I), \mathbf{D}(I)] .$$

其中：

## Semantic obligation load

$$\mathbf{S} = (m, \bar{a}, a_{\max})$$

其中：

$$m = |\mathcal{E}^S|$$

是 semantic obligation families 数，

$$a_j = |e_j|,$$
$$\bar{a} = \frac{1}{m} \sum_j a_j.$$

注意它不是 solver constraint row count。

一句：

“每个 customer 必须分配一次”

即使展开成十万条 rows，在 schema level 仍是一类 semantic obligation。

## Scope / scaffold structure

$$\mathbf{R} = (\bar{d}, d_{\max}, \mathbf{n}_{\text{scaffold}}).$$

其中  $d_j$  表示该 obligation family 涉及多少个 index dimensions，例如：

$$r(f, c, t) \Rightarrow d_j = 3.$$

而

$$\mathbf{n}_{\text{scaffold}}$$

记录：

- global;
- single-loop;
- nested;
- subset-indexed;
- temporal/recursive;
- cyclic;
- sliding-window;
- pairwise/all-pairs;
- extra scenario/commodity dimension。

这基本直接承接 MIPLIB-NL 的结构 taxonomy。 24

## Local coupling

核心只保留：



$$\mathbf{K}_L = (D_{\text{ov}}, J^+, \bar{J}).$$

它们很好解释，而且不需要复杂算法。

### Global coupling

核心：

$$\mathbf{K}_G = (\rho(W), C_{\text{spec}}, tw(G_P), C_{\text{tw}}).$$

辅助 diagnostics：

$$(C_{\text{hub}}, C_{\text{cyc}}).$$

第一版统计模型中我不会把 hubness、cycle density、spectral radius、treewidth 全部作为“同等独立 components”硬塞进去，因为它们很可能存在 multicollinearity。

### Dynamic / information structure

这一项不能完全由普通 hypergraph 捕获。

至少需要：

$$\mathbf{D} = (T_{\text{stage}}, N_{\text{state-link}}, N_{\text{scenario}}, D_{\text{nonanticipativity}}).$$

例如：

$$I_t = I_{t-1} + P_t - D_t$$

与独立 period constraints 的本质不同；stochastic optimization 中：

$$x_t^\omega = x_t^{\omega'}$$

在信息尚未分叉前的 non-anticipativity constraints 又进一步引入跨 scenario coupling。

这就是为什么 ORAgentBench 的“dynamic structure”和“constraint coupling”会发生 correlation：dynamic structure 本身经常就是制造 coupling 的 mechanism。 <sup>21</sup>

我们可以在定义中解决这个重叠：

**Dynamic structure 记录 coupling 的生成机制；coupling metrics 记录最终生成的 topology。**

它们就不再是同一概念。

### 为什么暂时不要直接量化 raw natural-language semantic coupling

你之前担心：

natural-language rules 中的 semantic coupling 是否能真正研究透、方便量化？

我的建议是：**不要把 raw linguistic semantic coupling 放在第一版核心。**

不要一开始试图用 embedding similarity:

$$\cos(e(r_i), e(r_j))$$

来定义约束 coupling。

“两个句子意思相似”与“两个 rules 共享 operational decisions”不是一回事。

最可靠的路线是：

|                                                                                                                                         |
|-----------------------------------------------------------------------------------------------------------------------------------------|
| NL rules $\rightarrow$ canonical semantic obligations $\rightarrow$ semantic entities $\rightarrow$ SOH $\rightarrow$ coupling metrics. |
|-----------------------------------------------------------------------------------------------------------------------------------------|

以后再研究 linguistic burden，例如：

- cross-sentence coreference;
- exception clauses;
- nested quantifiers;
- negation;
- conditional triggers;
- rules whose scope is defined by another rule;
- implicit business assumptions.

这些非常有意义，但 annotation noise 会高很多。

第一篇论文先把 **semantic structural coupling** 做扎实，成功概率明显更高。

**complexity 不能由 LLM accuracy 定义，但应该由 LLM accuracy 验证**

这一点我建议以后在论文 introduction 中说得非常清楚。

错误定义：

$$C(I) = 1 - \text{accuracy of GPT-X on } I.$$

因为换模型以后：

$$C(I)$$

就变了。

正确思路是：

$$C(I) = f(\text{problem structure}),$$

然后验证：

$$P(Y_{mi} = 1) = \sigma(\alpha_m - \beta^\top \mathbf{C}(I_i)),$$

其中：

- $m$ : LLM/agent;
- $i$ : problem;
- $\alpha_m$ : 模型能力;
- $\mathbf{C}(I_i)$ : problem intrinsic metrics。

一个真正好的 complexity measure 应该满足：

$$\beta_k > 0$$

在多数模型上方向一致，并且在 **unseen models**、**unseen benchmarks**、**unseen problem classes** 上仍能预测 performance degradation。

这比“某一模型 accuracy 和 complexity 有相关性”强很多。

还可以用多模型 response matrix：

$$Y_{mi} \in \{0, 1\}$$

通过 Rasch/IRT-style latent difficulty 得到一个 performance-derived item difficulty  $b_i$ ，但  $b_i$  只作为 **external criterion**：

$$\mathbf{C}(I_i) \longrightarrow b_i,$$

而不是拿  $b_i$  反过来定义 problem complexity。

## 将来怎样聚成 scalar

最初保留 vector。

若实验以后证明 components 稳定，可以学习：

$$D(I) = \sum_{k=1}^K w_k \tilde{C}_k(I), \quad w_k \geq 0, \quad \sum_k w_k = 1.$$

但 weights 必须在的一组模型上拟合：

$$\{M_1, \dots, M_r\},$$

然后在完全未参与 fitting 的：

$$M_{r+1}$$

和新 benchmark 上验证。

否则这个 scalar 只是“针对 GPT-X 拟合出来的 difficulty score”。

我暂时不推荐 PCA 作为最终定义，因为 PCA 最大化 variance，而不是 difficulty relevance，并且 component interpretation 会迅速变差。

## 完整例题：逐项计算 constraint coupling 和 complexity vector

下面刻意不用一个过于简单的 LP，而构造一个同时包含 facility activation、assignment、production、inventory、overtime、backlog 和 temporal ramping 的 **multi-period production-distribution problem**。

### 问题数据

设施：

$$F = \{A, B\}$$

客户：

$$C = \{1, 2, 3\}$$

period：

$$T = \{1, 2, 3\}.$$

Demand：

| customer | $t = 1$ | $t = 2$ | $t = 3$ |
|----------|---------|---------|---------|
| 1        | 30      | 35      | 25      |
| 2        | 20      | 25      | 30      |
| 3        | 25      | 20      | 30      |

base production capacities：

$$K_A = 50, \quad K_B = 45.$$

maximum overtime：

$$\overline{O}_A = 20, \quad \overline{O}_B = 15.$$

ramping：

$$R_A = 25, \quad R_B = 20.$$

production-on bounds：

$$M_A = 70, \quad M_B = 60.$$

facility fixed costs：

$$F_A = 100, \quad F_B = 80.$$

并给定 production、transport、inventory、backlog、overtime 的正常线性 costs。

## Canonical semantic decisions

定义：

$$y_f \in \{0, 1\}$$

facility 是否开放；

$$x_{fc} \in \{0, 1\}$$

customer 是否静态分配给 facility；

$$q_{ct} \geq 0$$

shipment；

$$p_{ft} \geq 0$$

production；

$$I_{ft} \geq 0$$

facility inventory；

$$o_{ft} \geq 0$$

overtime；

$$b_{ct} \geq 0$$

backlog；

$$u_{ft} \in \{0, 1\}$$

production-on decision。

因此 semantic entity families 是：

$$V^S = \{Y, X, Q, P, I, O, B, U\},$$

即：

$$|V^S| = 8.$$

grounded decision count 为：

$$|Y| = 2,$$

$$|X| = 2 \times 3 = 6,$$

$$|Q| = 2 \times 3 \times 3 = 18,$$

$$|P| = |I| = |O| = |U| = 2 \times 3 = 6,$$

$$|B| = 3 \times 3 = 9.$$

所以：

$$N_{\text{decision}} = 2 + 6 + 18 + 6 + 6 + 6 + 9 + 6 = 59.$$

这里已经可以看到为什么“59 variables”本身并不能描述问题：其中有些只是同一个 semantic family 的规则化展开；MIPLIB-NL 同样强调 industrial models 中的 solver-level rows 可以是少量 conceptual constraint families 经 loops 扩展出来的。 30

## Semantic obligations

我们定义十二个 obligation families。

$$r_1 : \quad \sum_f x_{fc} = 1 \quad \forall c$$

每个 customer exactly once:

$$e_1 = \{X\}.$$

$$r_2 : \quad x_{fc} \leq y_f$$

assignment only to open facility:

$$e_2 = \{X, Y\}.$$

$$r_3 : \quad q_{fct} \leq d_{ct} x_{fc}$$

shipment requires assignment:

$$e_3 = \{Q, X\}.$$

$$r_4 : \quad I_{ft} = I_{f,t-1} + p_{ft} - \sum_c q_{fct}$$

inventory balance:

$$e_4 = \{I, P, Q\}.$$

$$r_5 : \quad p_{ft} \leq K_f y_f + o_{ft}$$

capacity:

$$e_5 = \{P, O, Y\}.$$

$$r_6 : \quad o_{ft} \leq \overline{O}_f y_f$$

overtime only if open:

$$e_6 = \{O, Y\}.$$

$$r_7 : \quad b_{ct} = b_{c,t-1} + d_{ct} - \sum_f q_{fct}$$

demand/backlog balance:

$$e_7 = \{Q, B\}.$$

$$r_8 : \quad b_{ct} \leq \beta_c d_{ct}$$

service requirement:

$$e_8 = \{B\}.$$

$$r_9 : \quad p_{ft} \leq M_f u_{ft}$$

production activation:

$$e_9 = \{P, U\}.$$

$$r_{10} : \quad u_{ft} \leq y_f$$

production-on requires facility open:

$$e_{10} = \{U, Y\}.$$

$$r_{11} : \quad -R_f \leq p_{ft} - p_{f,t-1} \leq R_f$$

ramping:

$$e_{11} = \{P\}$$

在 schema graph 上是 unary, 但它带有 **temporal-coupled scaffold**。

最后:

$$r_{12} : \quad 100y_A + 80y_B + \sum_{f,t} o_{ft} \leq 230$$

global operating budget:

$$e_{12} = \{Y, O\}.$$

于是:

$$m = 12.$$

## Semantic load

hyperedge arities:

$$(1, 2, 2, 3, 3, 2, 2, 1, 2, 2, 1, 2).$$

总和:

$$23.$$

所以:

$$\bar{a} = \frac{23}{12} = 1.9167$$

和：

$$a_{\max} = 3.$$

semantic load：

$$\mathbf{S} = (12, 1.9167, 3).$$

### Scope/scaffold complexity

各 obligation 的 index dimension：

$$(1, 2, 3, 2, 2, 2, 2, 2, 2, 2, 0).$$

所以：

$$\sum_j d_j = 22$$

以及：

$$\bar{d} = \frac{22}{12} = 1.8333, \quad d_{\max} = 3.$$

但仅仅这个数依然漏掉  $r_4, r_7, r_{11}$  的 temporal recursion。

因此我们还应保留 scaffold labels，例如：

$$r_4, r_7, r_{11} \rightarrow \text{temporal/coupled},$$

$$r_3 \rightarrow \text{three-index nested},$$

$$r_{12} \rightarrow \text{global}.$$

这就是为什么 scaffold 应该作为 categorical structural annotation，而不是一开始随便给 nested=2、pairwise=4 这种未经验证的分数。MIPLIB-NL 同样把 scaffold 当作结构 taxonomy，而不是武断的单一 weighted score。 <sup>31</sup>

### Grounded semantic-obligation 数量

十二类 obligations 分别实例化为：

$$3, 6, 18, 6, 6, 6, 9, 9, 6, 6, 4, 1.$$

因此：

$$M_{\text{grounded}} = 80.$$



注意  $r_{11}$  每个 temporal pair 在 solver 中通常又对应上、下两条 inequalities, 所以 solver-level algebraic row count 会比 80 多。

这恰恰体现:

$$\text{semantic obligation count} /$$

= solver row c

## Mean overlap 的完整计算

十二项 constraints 共有:

$$\binom{12}{2} = 66$$

对。

其中有:

$$24$$

对满足:

$$e_i \cap e_j / \emptyset. =$$

所以:

$$D_{\text{ov}} = \frac{24}{66} = 0.3636.$$

几个具体例子:

$$J(r_1, r_2) = \frac{|\{X\}|}{|\{X, Y\}|} = \frac{1}{2}.$$

$$J(r_4, r_5) = \frac{|\{P\}|}{|\{I, P, Q, O, Y\}|} = \frac{1}{5}.$$

$$J(r_5, r_6) = \frac{|\{O, Y\}|}{|\{P, O, Y\}|} = \frac{2}{3}.$$

而:

$$e_6 = e_{12} = \{O, Y\},$$

所以:

$$J(r_6, r_{12}) = 1.$$

所有 66 对 Jaccard 的总和为:

$$\sum_{i < j} J_{ij} = \frac{281}{30} = 9.3667.$$

因此：

$$\bar{J} = \frac{9.3667}{66} = 0.14192.$$

条件 mean overlap：

$$J^+ = \frac{9.3667}{24} = 0.39028.$$

并验证：

$$0.3636 \times 0.39028 = 0.14192.$$

所以：

$$\mathbf{K}_L = (0.3636, 0.3903, 0.1419).$$

下面是这十二项 obligations 的 Jaccard overlap matrix：

Semantic obligation Jaccard overlap heatmap

这张图最重要的不是“颜色深就一定难”，而是可以直观看到 coupling 是 **clustered** 还是均匀分散。例如  $Y/O/P/U$  形成一个明显 operational subsystem，而  $Q/P/I/B$  形成另一个 subsystem，两者又经  $P$  等 entities 连接。

## Spectral coupling

构造：

$$W_{ij} = J_{ij}.$$

对上述矩阵求最大 eigenvalue 得：

$$\rho(W) = 2.01564.$$

由于：

$$m - 1 = 11,$$

所以：

$$C_{\text{spec}} = \frac{2.01564}{11} = 0.18324.$$

注意：

$$\bar{J} = 0.1419$$

而：

$$C_{\text{spec}} = 0.1832.$$

两者不同正是有意义的：后者不仅看平均 pair overlap，还受到 interaction topology 的影响。

## Treewidth

family-level primal graph 的 vertices:

$$\{Y, X, Q, P, I, O, B, U\}.$$

共同出现在 hyperedge 中的 entity pairs 给出 graph edges:

$$\begin{aligned} &(Y, X), (Y, P), (Y, O), (Y, U), \\ &(X, Q), \\ &(Q, P), (Q, I), (Q, B), \\ &(P, I), (P, O), (P, U). \end{aligned}$$

共:

$$11$$

条 edges。

对这个小 graph 可以精确枚举 elimination orders，得到：

$$\boxed{tw(G_P) = 2.}$$

例如一个 width-2 elimination ordering 是：

$$X, I, O, B, Q, Y, P, U.$$

因此：

$$\boxed{C_{\text{tw}} = \frac{2}{8-1} = 0.28571.}$$

这个结果其实非常有教育意义。

虽然：

- constraints 明显互相 coupling;
- overlap graph 中也存在很多 cycles;

但：

$$tw = 2$$

说明它仍然具有相当强的 global decomposability。

也就是说：

local overlap 高，并不必然等于 irreducible global interaction 高。

这正是保留多个 coupling diagnostics 而不是只用 constraint count 的价值。

## Hubness

各 semantic entities 被多少 obligation families 引用：

| $v$      | $Y$ | $X$ | $Q$ | $P$ | $I$ | $O$ | $B$ | $U$ |
|----------|-----|-----|-----|-----|-----|-----|-----|-----|
| $d_H(v)$ | 5   | 3   | 3   | 4   | 1   | 3   | 2   | 2   |

最大的是：

$$d_H(Y) = 5.$$

所以：

$$C_{\text{hub}} = \frac{5}{12} = 0.41667.$$

直观上，facility-open decision 是这个问题中的 major hub。

## Cycle density

obligation-overlap support graph：

$$q = 12,$$

$$M = 24,$$

并且整个 graph connected：

$$c = 1.$$

所以：

$$\mu = M - q + c = 24 - 12 + 1 = 13.$$

因此：

$$C_{\text{cyc}} = \frac{13}{24} = 0.54167.$$

这说明大量 overlap edges 不是维持 connectivity 所必需的，constraint interaction 中存在很多 alternative closed paths。

为了方便比较，下面把本例几个已经归一化到  $[0, 1]$  的 coupling diagnostics 放在一起。**这不是 aggregate complexity score，只是 visualization。**

Normalized coupling metrics

核心数值汇总：

| metric                           | value  | 直观意义                                    |
|----------------------------------|--------|-----------------------------------------|
| semantic obligation families $m$ | 12     | 有多少类独立 business requirements            |
| mean hyperedge arity $\bar{a}$   | 1.9167 | 一项 rule 平均同时涉及多少 decision types         |
| grounded decisions               | 59     | instance scale                          |
| grounded semantic obligations    | 80     | rules 实例化后的规模                           |
| mean index depth $\bar{d}$       | 1.8333 | rule family 的 indexing burden           |
| overlap density $D_{ov}$         | 0.3636 | 多少 rule pairs 发生直接 coupling             |
| conditional overlap $J^+$        | 0.3903 | coupled pair 平均共享程度                     |
| mean overlap $\bar{J}$           | 0.1419 | 总体 local coupling                       |
| spectral radius $\rho$           | 2.0156 | weighted global interaction strength    |
| normalized spectral coupling     | 0.1832 | relative spectral interaction           |
| treewidth                        | 2      | irreducible separator/interface size    |
| normalized treewidth             | 0.2857 | relative global coupling                |
| hubness                          | 0.4167 | 最大 semantic hub 覆盖比例                    |
| cycle density                    | 0.5417 | overlap network 闭合 interaction richness |

这个例子也说明一个非常重要的研究原则：

coupling is multidimensional.

一个 problem 可以：

- overlap density 高但 treewidth 低；
- hubness 高但 cycle density 低；
- constraint 数量很多但 schema complexity 很低；
- semantic obligation 很少但 pairwise scaffold 展开成  $O(n^2)$ 。

因此现在就把它们压成一个 scalar 会丢掉最有价值的信息。

## OR 问题类型、gold model 后的求解负担与核心假设

首先需要纠正一个 OR 分类上的常见混淆：

**LP、IP、MILP 是 mathematical formulation classes；scheduling、routing、facility location、inventory 等是 problem/application families。**

一个 scheduling problem 可以用 MILP、CP、CP-SAT、time-indexed formulation、disjunctive formulation 等不同 paradigm 表达。CP-Bench 就专门比较 MiniZinc、CPMpy 和 OR-Tools CP-SAT 三种不同抽象层次的 constraint modeling systems。 <sup>11</sup>

OPT-Engine 本身包括 inventory、portfolio、production、transportation、pollution control 五类 LP，以及 TSP、knapsack、bin packing、job-shop scheduling、minimum-cost network flow 五类 MIP tasks。

<sup>32</sup> MIPLIB-NL 又覆盖 assignment/resource allocation、cutting/packing、facility location、financial/revenue、network flow、production/scheduling、routing、graph-structured optimization 等更广类别。

<sup>33</sup> LEAN-LLM-OPT 则已经把 benchmark 扩展到 network revenue management、resource allocation、facility-location-type applications 和 airline network planning。 <sup>13</sup>

因此，以 **LLM+OR benchmark 的实验价值** 而不是整个 OR 学科的理论重要性排序，我建议你先掌握下表。

| Problem family                      | 典型问题                                                   | formulation 难在哪里                                          | gold model 后 solver code 是否标准              | gold model 后仍需明显算法设计?                                        | 对我们假设的意义                                                                                                                        |
|-------------------------------------|--------------------------------------------------------|-----------------------------------------------------------|--------------------------------------------|--------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Continuous LP / resource allocation | production mix、transportation、basic inventory          | indexing、balance、capacity、objective coefficients          | 很高；标准 LP API 足够                            | 低，通常是最干净的 control group                                      | 最可能支持 “modeling dominates” <sup>32</sup>                                                                                        |
| Generic MILP / selection / packing  | knapsack、assignment、bin packing                        | integrality、activation、logical implication、Big- $M$       | code 通常较标准                                 | 中；规模大时 search difficulty 可显著                                 | 第一个检验 “gold model $\neq$ automatic optimum” 的类别 <sup>34</sup>                                                                   |
| Scheduling                          | job shop、flow shop、crew/shift scheduling               | precedence、no-overlap、sequence、alternative machine、setup  | CP/MILP API 可以很标准，但 formulation choice 很重要 | 中到高；large instances 可能需要 CP/global constraints/decomposition | 很可能是 strong-hypothesis 的第一个边界；OR-Tools 的 job-shop formulation 就包含 precedence 与 no-overlap 两类核心约束 <sup>35</sup>                  |
| Routing                             | TSP、CVRP、VRPTW                                         | subtour、capacity、time windows、route consistency           | 小例可标准；工业规模不一定                              | 高；branch-and-cut、route generation、heuristics 经常重要            | gold conceptual model 给定后仍可能存在明显 computational burden；MIPLIB-NL 中 TSP 含 degree + subset-indexed subtour structure <sup>33</sup> |
| Network optimization                | shortest path/min-cost flow、多 commodity network design | flow conservation；multi-commodity 时出现共享 capacity coupling | pure flow 很高                               | pure network flow 低；multi-commodity/design 中高                | 很好的 matched comparison：语义相似但 coupling/solve burden 大不同 <sup>33</sup>                                                            |

| Problem family                  | 典型问题                                                       | formulation 难在哪里                                                     | gold model 后 solver code 是否标准 | gold model 后 仍需明显算法设计?                    | 对我们假设的意义                                                                                                                  |
|---------------------------------|------------------------------------------------------------|----------------------------------------------------------------------|-------------------------------|-------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| Facility location               | uncapacitated/ capacitated facility location               | open-assign implication、capacity、assignment                          | 很高                            | 小中规模 低—中；大规模 MILP 可高                      | 非常适合 controlled coupling experiments；MIPLIB-NL 将 assignment 与 capacity 描述为不同 nested families <span>30</span>              |
| Inventory / production planning | multi-period inventory、lot sizing                          | temporal balance、setup、capacity、cross-period coupling                | deterministic models 较标准      | deterministic LP 低；integer lot-sizing 中等  | 是研究“数量一样，但 temporal coupling 增加”的理想环境 <span>36</span>                                                                     |
| Revenue management              | network revenue management、choice-based allocation         | shared capacity、products/fare classes、choice models、network coupling | basic deterministic models较标准 | stochastic/ choice/network variants 可明显增加 | LEAN 的 Air-NRM 正好提供大规模、非教科书型 test bed <span>13</span>                                                                     |
| Stochastic/ dynamic OR          | scenario planning、multi-stage allocation、online replanning | non-anticipativity、states、scenario trees、information timing          | 低于简单 LP/ MILP                 | 高，常伴随 decomposition/ rolling horizon      | 很可能必须保留 independent computational/ dynamic axes；ORAgentBench 已包含 stochastic 与 multi-step replanning tasks <span>37</span> |

这里最值得你抓住的是：

“solver code 标准化” /

= “求解过程

写：

model.optimize()

可以只有一行代码。

但 solver 在这一行后面做的 branch-and-bound、cuts、presolve、heuristics、decomposition-like internal procedures 可能非常复杂。Gurobi 的官方接口明确记录 MIP best bound、incumbent、gap、runtime 和 deterministic work，并允许按 time、node、gap 等条件提前终止。 38

所以你原来的直觉最好改写为：

**从 gold mathematical model 到 syntactically correct solver API code 的 translation error 很可能很低；但从 gold model 到 certified optimum 的 residual computational difficulty 不一定低。**

这两个 “model→solver” 必须拆开。

这也是为什么我非常赞成你的 gold-model experiment，但实验不能只问：

“LLM 能不能把 gold model 写成 Gurobi code？”

而要进一步问：

“正确 code 在 standardized solver budget 下是否能稳定得到 feasible / near-optimal / certified-optimal solution？”

## 验证实验设计：如何证明 complexity 真有预测价值

### 最关键的 gold-model intervention ladder

这是整篇论文最重要的 causal experiment。

对同一个 instance  $I_i$ ，做四种 intervention。

#### Full operational input

$$A : (\text{NL} + \text{data} + \text{business rules}) \rightarrow \text{agent} \rightarrow \text{solver}.$$

这是现实 baseline。

#### Gold semantic layer

$$B : (\text{gold canonical entities} + \text{gold semantic obligations} + \text{data}) \rightarrow \text{agent} \rightarrow \text{formulation/code} \rightarrow \text{solver}.$$

这一层消除了大部分 natural-language grounding ambiguity，但仍然要求模型决定如何把 obligations mathematicalize。

#### Gold mathematical formulation

$$C : M_i^* \rightarrow \text{LLM} \rightarrow \text{solver code} \rightarrow \text{solver}.$$

这一层直接测试你的核心假设：

correct formulation→solver code 是否基本不会再失败。



## Gold executable model

$D$  : gold executable solver code  $\rightarrow$  solver.

这一层完全拿走 LLM formulation/code generation, 剩下:

$$C_{\text{solve}}.$$

于是定义:

$$p_A, p_B, p_C, p_D$$

分别表示各阶段成功率。

进一步:

$$\Delta_{\text{NL/sem}} = p_B - p_A$$

衡量 semantic grounding contribution;

$$\Delta_{\text{form}} = p_C - p_B$$

衡量 mathematical representation contribution;

$$\Delta_{\text{code}} = p_D - p_C$$

衡量 model-to-code contribution;

$$\epsilon_{\text{solve}} = 1 - p_D$$

衡量 residual solver failure。

如果对 LP、standard MILP、facility location 等 target benchmark:

$$p_C \approx p_D \approx 1$$

且:

$$p_A \ll p_B < p_C,$$

那么你就获得非常漂亮的证据:

$$\text{semantic/formulation difficulty dominates.}$$

反之, 如果 scheduling/routing/large MIP 上:

$$p_D < 1$$

或者:

MIP gap remains large,

那么这不是“实验失败”，反而会得到更有价值的结论：

OPC genuinely has two separable axes: modeling complexity and residual solve complexity.

这比预设“computational complexity 不重要”更强。

## 哪些 benchmark 最适合这个实验

**MIPLIB-NL 是第一优先级。**

它直接提供 natural-language description、formal mathematical model 和 raw data，而且基于真实 MIPLIB 2017 models 做一一结构恢复；因此天然适合 A/C/D intervention。<sup>22</sup> 更重要的是，它已经显式分析 semantic types 与 loop scaffolds，因此可以直接给 OMC annotation 提供起点。<sup>30</sup>

**OPT-Engine 是第二优先级。**

因为它的 generator 与 solver template 对齐，可以精确控制 problem scale 和 constraint augmentation；而且它已经发现增加少量 noncanonical constraints 会产生显著 accuracy degradation，因此非常适合制造：

same scale + different coupling

的 controlled pairs。<sup>12</sup>

**CP-Bench 是第三优先级。**

它有 runnable CP models，并覆盖不同 constraint modeling frameworks，特别适合验证我们的结论是否只是 MILP/Gurobi artifact。<sup>11</sup>

**CIR/R2C 的 ORCOpt-Bench 非常适合 semantic-layer intervention。**

它正是围绕 complex operational rules 和 rule-to-constraint mapping 构造，因此和我们的 Semantic Obligation 层高度互补。<sup>15</sup>

**LEAN Large-Scale-OR 非常适合测试 scale confounding。**

它有 101 个 problem instances，明确包含 medium/large output models，并观察到 input length 与 output-variable count 增大时 modeling performance 下降。<sup>13</sup> 这使我们可以检验：

SOH coupling metrics

能否在控制：

input tokens, #variables, #constraints

以后仍然解释额外 variance。

**ORAgentBench 更适合作为 external validity，而不是第一版 gold-model intervention dataset。**

因为它的价值在完整 operational artifact→decision workflow，并有 hidden validators；它很好地测试我们的 complexity 能否预测真正 end-to-end failure，但公开 gold formulation 的便利程度不如 MIPLIB-NL。

ORAgentBench 的 agent failures 包括 missed operational rules、brittle formulations、weak feasible-solution construction 和 inadequate improvement strategy，正好可以验证我们的 modeling 与 solving axes 是否能够分离。 39

**ConstraintBench 则作为 direct-solving contrast group。**

它完全不允许 solver，200 个 tasks 横跨十个 OR domains；最佳模型只有 65% feasibility，而 feasible outputs 通常已经达到 Gurobi optimum 的约 89–96%，joint feasibility + near-optimality 最高只有 30.5%。

17 这非常强烈地说明：

$$P(\text{feasible})$$

与：

$$P(\text{near-optimal} \mid \text{feasible})$$

应该分开评估。

**feasibility 与 optimality 一定要彻底拆开**

对每个 agent output，至少保留：

$$F_i = \mathbf{1}[\text{all hard constraints satisfied}],$$

然后：

$$V_{\text{count}} = \#\{\text{violated obligation families}\},$$

以及 normalized violation magnitude：

$$V_{\text{mag}} = \sum_j w_j \frac{\max(0, g_j(x))}{s_j + \epsilon}.$$

optimality 则只在 feasible condition 下分析：

$$G_i = \frac{|z_i - z_i^*|}{|z_i^*| + \epsilon}.$$

因此核心结果应该是两条曲线：

$$P(F = 1 \mid \mathbf{C})$$

和：

$$E[G \mid F = 1, \mathbf{C}].$$

不要只报告一个 end-to-end exact accuracy。

这不仅因为 ConstraintBench 已经发现 feasibility/optimality decoupling；LEAN-LLM-OPT 也观察到 **结构错误的 formulation 有时仍然能碰巧得到正确 optimal value**，因此只比较最终 objective value 甚至可能把错误模型判成正确。 40

所以模型层还需要：

constraint-family recall,  
constraint-family precision,  
variable/entity alignment,  
objective correctness,

以及可用时的 mathematical-equivalence check。OptiBench/ORGEval 类工作提供了 formulation-equivalence evaluation 的直接先例。 <sup>10</sup>

## 最强的 complexity validation 不是相关性，而是 controlled intervention

仅仅在现有 benchmark 上做：

$$\text{corr}(\bar{J}, \text{accuracy})$$

是不够的。

因为：

$$\bar{J}$$

可能和：

$$\#variables, \#constraints, \text{tokens}, \text{problem class}$$

一起增长。

真正漂亮的实验是生成 **matched counterfactual pairs**。

例如固定：

$$|V|, \quad |\mathcal{E}|, \quad \text{constraint types}, \quad \text{coefficient distribution},$$

只改变 topology。

低 coupling：

$$e_1 = \{x_1, x_2\}, e_2 = \{x_3, x_4\}, e_3 = \{x_5, x_6\}.$$

高 coupling：

$$e_1 = \{x_1, x_2\}, e_2 = \{x_2, x_3\}, e_3 = \{x_1, x_3\}.$$

两组：

$$\#constraints = 3$$

完全一样，但 coupling 完全不同。

更进一步，可以构造：

path  $\rightarrow$  cycle  $\rightarrow$  hub  $\rightarrow$  clique

系列，同时尽量保持 semantic atom 数量和 token length 相同。

于是直接测试：

Does LLM feasibility decrease monotonically as coupling topology increases?

这是比 OPT-Engine 的 “add constraints” 更强的 intervention，因为你可以让：

$$\Delta \#constraints = 0$$

而只改变：

$$\Delta tw, \quad \Delta \rho, \quad \Delta \bar{J}.$$

如果 accuracy 仍下降，你就真正获得了：

**coupling—not constraint count—is a source of modeling difficulty**

的证据。

## metric 是否互相干扰，怎样检查

第一步先做 correlation matrix：

$$corr(\bar{J}, D_{ov}, \rho, tw, C_{hub}, C_{cyc}).$$

我预计：

$$\bar{J} \leftrightarrow \rho$$

以及：

$$C_{hub} \leftrightarrow \rho$$

会有明显 correlation。

不要因为相关就删掉，而要问：

它们是否有 incremental explanatory value?

例如建立：

$$\text{Model 0 : } Y \sim \#variables + \#constraints + \text{tokens}.$$

然后：

$$\text{Model 1 : } Y \sim \text{baseline} + \bar{J}.$$

然后：

$$\text{Model 2 : } Y \sim \text{baseline} + \bar{J} + tw.$$

最后：

$$\text{Model 3 : } Y \sim \text{baseline} + \bar{J} + tw + \rho.$$

比较 held-out predictive likelihood/AUC，而不是只看 in-sample  $R^2$ 。

如果：

$$\rho$$

在已有：

$$\bar{J}, tw$$

以后几乎没有 incremental gain，就把 spectral metric 降级为 diagnostic。

这就是解释性优先于“指标越多越好”。

### **manual annotation 应该从很小的核心集开始**

第一版 pilot 不需要给几千道题全部人工标注。

最合理的是先选择约：

60–80

个 instances，覆盖：

- LP/resource allocation;
- facility location;
- network flow;
- scheduling;
- routing;
- inventory/lot-sizing;
- revenue/stochastic 少量实例。

两个有 OR 基础的 annotators 独立标：

semantic entities,  
obligation boundaries,  
hyperedge scopes,  
scaffold type.

然后用 adjudicator 解决 disagreement。

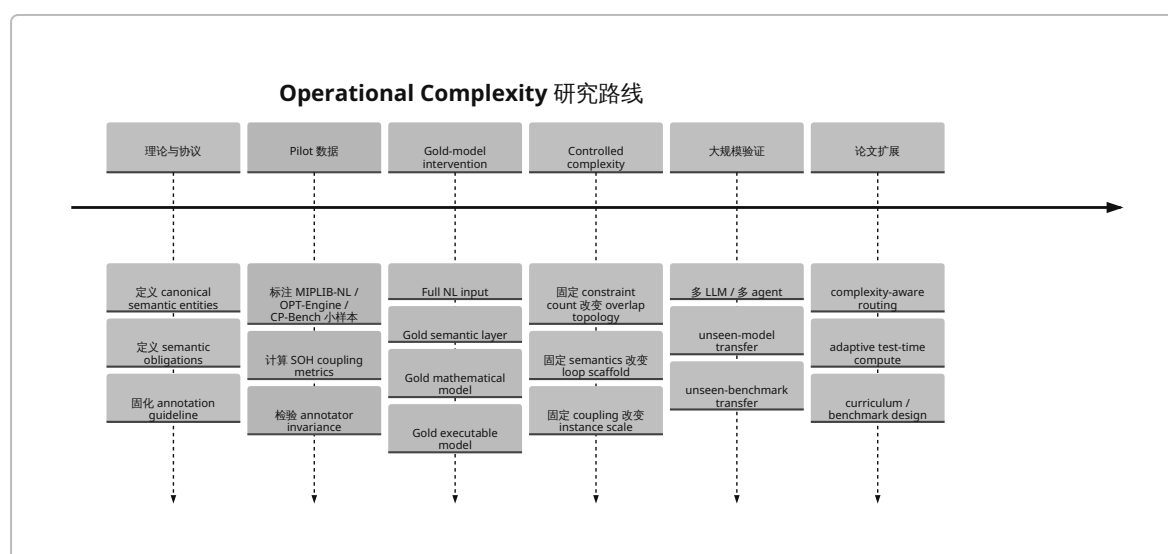
第一轮不要要求 annotator 人工计算 spectral radius/treewidth；只标 semantic structure, metrics 自动计算。

annotation reliability 可以分别评估：

entity matching F1,  
hyperedge-scope Jaccard,  
scaffold categorical agreement.

如果这一步 reliability 做不起来，就说明 canonical semantic layer 还没有定义清楚，应该先解决 ontology，而不是继续训练模型。

## 整体实验路线



## 论文级研究路线与最终建议

### 第一篇论文真正应该宣称什么

我不建议第一篇就宣称：

“We define operational complexity.”

这个 claim 太大，也容易遭到 reviewer 追问：“为什么你的几个 graph metrics 就是 operational complexity?”

更稳而且更有力量标题方向是：

**Towards a Semantic-Structural Theory of Operational Modeling Complexity**

或：

**Measuring Operational Modeling Difficulty in LLM-based Optimization through Semantic Constraint Structure**

核心 contribution 可以明确成：

Representation + Metrics + Controlled Benchmark + Causal Intervention

即：

**第一，提出 problem-oriented semantic representation。**

把 operational problem 从 solver-level algebraic model 中抽离出来：

$$I \rightarrow \text{canonical semantic entities} + \text{semantic obligations} + \text{scaffolds}.$$

这是 SOH。

**第二，提出一组有理论来源且可解释的 complexity descriptors。**

最核心保留：

$$m, \bar{d}, D_{\text{ov}}, \bar{J}, \rho, tw.$$

其中 treewidth/hypergraph ideas 有 CSP 理论基础，而 semantic/scaffold decomposition 有 MIPLIB-NL 和 CIR 等最新 OR modeling work 的直接背景。<sup>41</sup>

**第三，不用 observational correlation 作为唯一证据，而设计 topology-controlled instances。**

这会把论文从：

“我们发现 graph feature 和 accuracy 有 correlation”

提升成：

“在控制 constraint count、problem size 和 semantic types 后，独立改变 coupling structure 会系统性改变 LLM modeling failure。”

这是明显更强的 scientific contribution。

**第四，用 gold-model intervention 给 modeling complexity 与 computational burden 做 stage decomposition。**

这直接回答你最开始最重要的问题：

Where does operational difficulty actually enter the LLM+solver pipeline?

OPT-Engine 已经提供 constraint bottleneck 的初步证据，而它也主动承认 exact-solver conclusion 不应无条件推广到 large-scale NP-hard regimes。<sup>42</sup> 我们的工作可以把这从 benchmark-specific observation 变成 **cross-class causal decomposition**。

**我目前最推荐的 metric 优先级**

在数学基础和 annotation budget 都有限的条件下，我会这样排序：



| 优先级        | metric                        | 原因                                                 |
|------------|-------------------------------|----------------------------------------------------|
| 核心         | semantic obligation count $m$ | 最直接、最稳定、区别于 solver rows                            |
| 核心         | index/scaffold structure      | MIPLIB-NL 已经给出非常直接的文献基础 <sup>27</sup>              |
| 核心         | overlap density $D_{ov}$      | 最容易解释                                              |
| 核心         | mean Jaccard $\bar{J}$        | 直接表示 shared semantic entities                      |
| 核心         | treewidth $tw$                | 理论基础最强，代表 global decomposability <sup>26</sup>     |
| 次核心        | spectral radius $\rho$        | 捕获 global weighted connectivity，但需检验是否提供增量信息       |
| diagnostic | hubness                       | failure localization 很直观，但容易与 spectral coupling 重合 |
| diagnostic | cycle density                 | 图结构解释好，但是否预测 LLM failure 尚未知                       |
| 后续         | fractional hypertree width    | 理论更忠实于高-arity constraints，但第一篇实现负担较大 <sup>26</sup> |
| 后续         | raw NL semantic features      | 有潜力，但 annotation / parser noise 最大                 |

换句话说，第一篇不要追求“数学越复杂越好”。

**一项 treewidth + 两项 overlap + 一项 scaffold taxonomy，如果能够稳定预测 cross-model failure，就已经非常有价值。**

## 最重要的 falsifiable hypotheses

论文最好围绕几条可以被证伪的 hypothesis，而不是围绕一个先验正确的定义。

$H_1$  : Semantic-structural metrics predict LLM modeling failure beyond size proxies.

即控制：

$\#variables, \#constraints, tokens$

以后：

$\bar{J}, tw, scaffold$

仍然显著。

$H_2$  : Coupling predicts feasibility failure more strongly than optimality loss conditional on feasibility.

这是你关于 feasibility 更难的 intuition，而且 ConstraintBench 已提供与之相容的独立 evidence. <sup>17</sup>

$H_3$  : Giving gold semantic obligations produces a larger performance gain than giving additional generic reason

这可以直接测试 semantic grounding 是不是 root bottleneck。

$$H_4 : P(\text{success} \mid \text{gold mathematical model}) \gg P(\text{success} \mid \text{full NL instance})$$

在 exact-friendly classes 中成立。

$$H_5 : \text{Residual computational burden remains non-negligible in large routing/scheduling/MILP instances.}$$

如果成立，就界定 OPT-Engine strong statement 的 applicability boundary，而不是推翻 OPT-Engine。OPT-Engine 自身对 large-scale NP-hard exact solving 的 limitation 已作出同方向说明。 <sup>3</sup>

$$H_6 : \text{Complexity ranking transfers across LLMs.}$$

这是 “problem intrinsic” claim 最关键的 empirical test。

假如 GPT-A 认为：

$$I_1 < I_2 < I_3$$

而 GPT-B、GPT-C 大体也是：

$$I_1 < I_2 < I_3,$$

才有资格说我们发现的是 problem difficulty，而不仅是某个 model 的 idiosyncratic weakness。

## operational complexity 还能形成哪些后续论文

定义只是开始。

最自然的第二步是 **complexity-aware agent routing**：

$$\mathbf{C}(I) \rightarrow \begin{cases} \text{direct solver formulation} \\ \text{multi-agent formulation} \\ \text{decomposition} \\ \text{CP formulation} \\ \text{heuristic search} \\ \text{human escalation} \end{cases}$$

ORAgentBench 已经明确指出真实 agent 不仅需要 modeling strategy，还要选择 exact、heuristic、repair、decomposition 等 solving strategies。 <sup>43</sup> 所以 complexity vector 可以从 “evaluation metric” 进一步变成 **policy variable**。

另一个方向是 **complexity-aware test-time compute allocation**：

$$\text{budget}(I) = g(\mathbf{C}(I)).$$

低 coupling problems 单次 formulation；

高 treewidth / high semantic coupling problems 自动增加：

- independent formulation samples；
- constraint-by-constraint verification；

- critic agents;
- solver-feedback iterations;
- decomposition attempts。

这样 complexity 不再只是描述性指标，而能产生 performance improvement。

第三个方向是 **complexity-controlled benchmark generation**。OPT-Engine 已经证明 controllable scaling 对研究模型能力边界非常 valuable；MIPLIB-NL 又表明仅改变 semantic type 不足，需要保留 loop/scaffold structure。<sup>44</sup> 我们可以进一步构造：

same size, different coupling

以及：

same coupling, different scale.

这会是一个非常干净的 benchmark contribution。

第四个方向是 **failure localization**。若模型漏掉一个 obligation  $r_j$ ，可以分析它在 SOH 中的：

$d(r_j)$ ,  $\text{centrality}(r_j)$ ,  $\text{hub exposure}$ ,

看 high-centrality obligations 是否更容易造成 cascading model failure。

第五个方向是 **complexity-calibrated confidence**：

$P(\text{model correct} \mid \mathbf{C}, I, M)$ .

在 industrial deployment 中，模型不仅要给 answer，还需要知道什么时候应该 abstain 或要求 human review。这与 ORAgentBench 所强调的 dependable operational decisions 而不只是 plausible code 高度一致。<sup>45</sup>

## 我认为目前最关键的理论结论

经过这轮 literature review，我会把我们接下来要建立的理论中心压缩成下面一句话：

**Operational Modeling Complexity** should measure not how many constraints a formulation contains, but how many independent semantic obligations an operational instance imposes, how those obligations are instantiated across operational scopes, and how strongly they interact through shared decision entities and global dependency structure.

中文就是：

**Operational Modeling Complexity** 不应回答“模型有多大”，而应回答“为了完整地保持这个 operational problem 的逻辑，建模者必须同时维护多少种语义义务、这些义务如何跨 index/scenario/time 展开，以及它们之间形成了多难被局部分解的耦合结构”。

在这个框架下：

$\#constraints$

只是规模；

semantic obligation count

表示 rule load；

scaffold

表示 rule 如何展开；

$\bar{J}$

表示 local coupling；

$\rho$

表示 weighted global reinforcement；

$tw$

表示 global decomposability；

dynamic structure

表示 coupling 如何沿 time/scenario 传播；

而：

$C_{\text{solve}}$

则被故意隔离出来，用 gold-model intervention 判断它究竟能不能在目标 LLM+OR regime 中忽略。

这比一开始把：

$$\text{operational complexity} = \alpha \#constraints + \beta \#variables$$

或者：

$$\text{operational complexity} = 1 - \text{LLM accuracy}$$

都更接近你最初想类比 computational complexity 的目标。

最有价值的是，这个定义是 **可被证伪的**：如果 controlled experiments 发现 treewidth、overlap、scaffold 在控制规模之后完全不能预测 LLM degradation，我们就必须修改理论；如果它们可以跨模型、跨 benchmark 稳定预测 failure，那么才逐步获得把这个 vector 称为 **operational modeling complexity measure** 的资格。

这也是我认为目前最有可能从“一个有趣想法”发展成顶会/顶刊研究的路线。

- 1 2 3 12 32 34 42 44 <https://arxiv.org/html/2601.19924v2>  
<https://arxiv.org/html/2601.19924v2>
- 4 <https://docs.gurobi.com/projects/optimizer/en/current/features/solutionpool.html>  
<https://docs.gurobi.com/projects/optimizer/en/current/features/solutionpool.html>
- 5 26 41 <https://arxiv.org/abs/0911.0801>  
<https://arxiv.org/abs/0911.0801>
- 6 <https://aclanthology.org/2022.emnlp-industry.4.pdf>  
<https://aclanthology.org/2022.emnlp-industry.4.pdf>
- 7 <https://arxiv.org/abs/2402.10172>  
<https://arxiv.org/abs/2402.10172>
- 8 <https://arxiv.org/abs/2405.17743>  
<https://arxiv.org/abs/2405.17743>
- 9 <https://arxiv.org/html/2508.10047v1>  
<https://arxiv.org/html/2508.10047v1>
- 10 <https://openreview.net/forum?id=KD9F5Ap878>  
<https://openreview.net/forum?id=KD9F5Ap878>
- 11 <https://arxiv.org/abs/2506.06052>  
<https://arxiv.org/abs/2506.06052>
- 13 <https://arxiv.org/html/2601.09635v2>  
<https://arxiv.org/html/2601.09635v2>
- 14 23 24 25 27 30 31 33 36 <https://arxiv.org/pdf/2602.10450>  
<https://arxiv.org/pdf/2602.10450>
- 15 <https://arxiv.org/abs/2602.02029>  
<https://arxiv.org/abs/2602.02029>
- 16 19 20 21 37 39 43 45 <https://arxiv.org/html/2606.19787>  
<https://arxiv.org/html/2606.19787>
- 17 40 <https://arxiv.org/abs/2602.22465>  
<https://arxiv.org/abs/2602.22465>
- 18 <https://miplib.zib.de/>  
<https://miplib.zib.de/>
- 22 <https://arxiv.org/abs/2602.10450>  
<https://arxiv.org/abs/2602.10450>
- 28 <https://arxiv.org/html/2004.13491v1>  
<https://arxiv.org/html/2004.13491v1>
- 29 38 <https://docs.gurobi.com/projects/optimizer/en/current/concepts/parameters/guidelines.html>  
<https://docs.gurobi.com/projects/optimizer/en/current/concepts/parameters/guidelines.html>
- 35 [https://developers.google.com/optimization/scheduling/job\\_shop](https://developers.google.com/optimization/scheduling/job_shop)  
[https://developers.google.com/optimization/scheduling/job\\_shop](https://developers.google.com/optimization/scheduling/job_shop)